COMPUTABILITY / RESEARCH REPORT

PA194R

A different 194-state machine calibrated to the consistency of PA

Comparison of the supplied designs, a new literal decoder, an ordinary PA proof, and reproducible execution certificates

Abstract

No verified machine with fewer than 194 working states was found. The main fallback, PA194R, combines a new decoder association with a known-zero closing decrement in the symmetric cached controller. Its 1,111 changed source-address paths implement exactly the resolved source program of the simpler new table PA194F. A four-site invariant proves that program equivalent, step for step, to the supplied PA194CS source. Complete literal checks cover all 4,746 primitive outcomes, including the unreachable positive decrement branch; blank startup reaches the clean source seed after 57,768 transitions.

The zero-closing principle already occurs in the supplied 196-state research. The new contribution is the verified 194-state composition, its different decoder graph, the small preservation proof and the bindings to the exact binary tables. The ordinary conclusion is

$$\mathrm{PA} \vdash \neg\,\mathrm{Halt}(\mathrm{PA194R}) \leftrightarrow \mathrm{Con}(\mathrm{PA}).$$

The inherited proof-producing logical calibration is stated and attributed explicitly. This is not an end-to-end Lean or Rocq proof.

14 November 2026

Pinned ProveIt revision:

`17f92105e2bc4317feae6c219cb2143437b482ff`

The archive includes four new tables (R, F, A and X), source graphs, builders, independent symbolic checks, certificates, comparison data, negative controls and the source of this report.

Contents

1 Result and exact scope **2**

2 Comparison of the supplied research **2**

2.1 Snapshot and deduplication . . . 2

2.2 The eight existing 194-state graph classes . . . 3

2.3 How the larger designs lead to the frontier . . . 3

3 The source change isolated in PA194F **3**

3.1 Assembly and count . . . 3

3.2 Exact edit . . . 4

3.3 Clean seed and tape representation . . . 4

4 The inherited PA calibration and its audit **5**

5 The new source transformation and its PA proof **7**

5.1 The exact four-site invariant . . . 8

5.2 Binding the source proof to the binary table . . . 9

6 PA194R: a reassociated 194-state decoder **10**

6.1 Exact source equality and independent literal verification . . . 11

7 Two companion 194-state compositions **12**

8 Verification: what was established **12**

8.1 Two implementations and their boundaries . . . 12

8.2 A useful rejected construction . . . 13

9 Search below 194 states **13**

10 Ideas from ZFC and ConciseIPRA **14**

11 Using and reproducing the archive **14**

A Complete dense transition table for PA194R **16**

B All 41 research packets in the comparison **17**

1 Result and exact scope

The target was a binary blank-tape machine below 194 states with a PA-provable nonhalting equivalence to Con(PA), or, failing that, a materially different 194-state design. The first target remains open after the searches recorded here. The primary fallback is PA194R: it changes the decoder association at 1,111 source paths and replaces the axiom emitter's final unconditional return by a register operation whose correctness requires a new invariant. Its literal graph is not among the eight prior 194-state graph classes, even after arbitrary state renaming, a different root, tape reflection or global bit exchange.

The simpler PA194F isolates the source change in one native decoder row of PA194CS. Its extra positive branch has a different source successor and is excluded by proof, rather than being silently removed. PA194R then implements exactly this new source graph with a different 67-reader grammar and compatible fold set. Two companion tables, PA194A and PA194X, change a separate entry jump. All four have complete literal tables and reproducibility artifacts.

The novelty is deliberately characterized narrowly: a new source/decoder composition and layout, not a new arithmetization of PA or a state-count improvement. Prior 196-state zero-closing work is credited in Section 5. The searches do not establish that 194 is minimal.

All delivered machines use a two-way infinite tape with alphabet $\{0, 1\}$, initially all zero, a head initially at coordinate zero, and only left or right moves. Dense files use working controls $0, \ldots, 193$, start control 0, and a unique halt label 194 with no outgoing row. The halt label is not counted. Each table has exactly 388 rows. Native files preserve compiler labels: start 74, source dispatch 84, and halt 254.

Let H_M be the arithmetic sentence asserting a finite halting trace of the specified table from this exact blank configuration. For an ordinary primitive-recursive PA proof predicate put

$$\mathrm{Ref}_{\mathrm{PA}} = \exists p \, \mathrm{Prf}_{\mathrm{PA}}(p, \ulcorner 0 = 1 \urcorner), \qquad \mathrm{Con}(\mathrm{PA}) = \neg \, \mathrm{Ref}_{\mathrm{PA}} \, .$$

The asserted theorem is an internal PA biconditional. Its proof does not assume consistency. Actual standard-model nonhalting follows if PA is consistent. Under that usual assumption PA cannot prove this particular nonhalting sentence, by its second incompleteness theorem and the displayed equivalence.

2 Comparison of the supplied research

2.1 Snapshot and deduplication

The comparison is tied to revision `17f92105e2bc4317feae6c219cb2143437b482ff` of the user's ProveIt repository [1]. It covers the PA research tree and every current archive under `docs/incoming`, including nested archives. The report ledger contains 36 relocated research packets and five newer incoming packets: 41 packages. The 26 current incoming ZIP files overlap the relocated packets; they are not 26 additional independent designs.

The literal-table recognizer accepts complete numeric five-column two-symbol L/R tables with one undefined halt target in `.tm`, `.txt`, `.tsv` or `.csv` files. It finds 4,417 occurrences, 620 distinct byte strings and 446 root-spanning control-graph classes. Two disconnected synthetic controls are excluded from the last canonicalization. These totals include rejected candidates, partial compilations and test controls, not just valid consistency machines. JSON-only and source-only encodings are outside this literal-table count; the packet audit still records their reported designs. Within the recognized tables there are no 10–193-state tables; six- and nine-state tables are synthetic controls.

Byte hashes, graph hashes, archive hashes and occurrence paths are retained in `comparison/census/`. Appendix B lists all 41 packages. This audit compares their claims, architecture and supplied evidence; it does not claim to have independently reproved every old candidate.

2.2 The eight existing 194-state graph classes

The 97 occurrences of 194-state tables collapse to 15 byte strings and eight control-graph classes. Seven are proposed or supported designs; one is an explicitly rejected scanner quotient. The different recent PA194C archives repeat the same principal table [2, 3].

Design	Sites	Outcomes	Distinguishing feature and evidence
PA194–67/7	3,673	4,780	Separate-head source; $134 + 67 - 7 = 194$. Two symbolic physical implementations supplied.
PA194–AD	3,673	4,780	Same source, physical A/D exchange. Two implementations supplied.
PA194–68/8	3,673	4,780	Different decoder association; $134 + 68 - 8 = 194$. Main build freshly rerun successfully.
PA194C	3,649	4,749	Cached-head Shift/MP controller; repeated principal table in newer packets. Full supplied physical checks.
PA194CS	3,646	4,745	Symmetric destructive comparison and fold $41 \mapsto 330$. Supplied source and physical checks.
PA194CZ	3,649	4,750	Source 26 becomes a zero-register decrement. Construction/invariant evidence; no separate independent literal replay in its packet.
PA194altC	3,649	4,749	Cached controller on the 68-reader/eight-fold decoder. Reproducible checked search input.
Rejected scanner	3,700	—	Local folds pass, but a seven-zero source address fails the literal scanner contract at source 1003.

Table 1: Distinct prior 194-state graph classes. Sites count primitive source instructions, not physical controls. Evidence is supplied unless a fresh rerun is explicitly stated. Exact identities are in `comparison/frontier194.csv`.

The accepted 68/8 design and rejected 68/8 scanner candidate must not be conflated. The former has the separate 190-token emitter and 3,673 sites. The latter uses the diagonal 191-token emitter and 3,700 sites; its address scanner fails despite a plausible state count.

2.3 How the larger designs lead to the frontier

The architecture is a register program compiled through a binary source tree into a tape interpreter and an axiom decoder. Useful comparisons therefore have three separate metrics: primitive source length, distinct decoder subtrees, and folds that can be applied simultaneously to the literal table.

The root PA README distinguishes ordinary mathematical claims, standard finite-run Lean results, conditional arithmetic endpoints, and unfinished machine-specific iPRA reductions [1]. The imported PA196 formal result gives literal nonhalting implies PA consistency; that alone does not establish the PA biconditional. The proof below uses the retained ordinary proof-producing calibration, not a claim that the repository's iPRA objective is complete.

3 The source change isolated in PA194F

3.1 Assembly and count

The builder reconstructs the inherited fixed interpreter and source nodes, applies the cached controller and symmetric guard, and compiles the frozen 67-reader axiom grammar. PA194F changes only the final axiom leaf from `breakTo1` to `dec6`. The selector for register 6 already exists; no new control is needed.

Family	Comparison and implication
U208 and predecessors	Conditional entry contexts, finite history and three-cell folds reduce decoder controls. Several later packets report bounded searches without an accepted 207-state result. Report names and counts do not constitute independent logical calibrations.
PA196	The inherited 201-token axiom emitter expands to 3,812 sites and 4,959 outcomes. Its register engine underlies the later family. Same-size alternatives vary association, layout, endpoint conventions and zero-closing returns.
Diagonal 191	The diagonal-vacuity calculus gives a 191-token emitter and 3,700 sites. Original PA195 uses $134 + 68 - 7 = 195$; scanner-eight M195S8 uses $135 + 68 - 8 = 195$.
Separate 190	Separating the diagonal head saves another token and gives 3,673 sites. PA195S and both original PA194 allocations compile this source. Calibration accounts for its changed ten-entry interface.
Cached controller	The Combined/C change removes 24 sites. Shift checks the cached theorem; successful MP leaves its consequent in the cache, clears top and offset, and avoids a redundant stack operation. PA194CS saves three more sites by symmetry.
NEG18	An 18-instruction negation macro reduces the source to 3,441 sites in the recorded experiment but compiles to 197 states. Source shortening alone is not a physical state-count improvement.

Table 2: Architectural comparison of larger supplied designs. The packet ledger gives per-report provenance.

The raw machine has 202 controls: 135 fixed controls and 67 axiom readers. The seven inherited decoder folds and the symmetric-guard fold $41 \mapsto 330$ are disjoint and apply simultaneously:

$$135 + 67 - 8 = 194.$$

All 32 two-bit local history cases are checked on the simultaneously folded table. Full source-instruction checks then run on the final quotient. There are 3,646 source sites and 4,746 outcomes; the extra outcome relative to PA194CS is the new decrement's positive branch. All working controls are graph-reachable from the start, and all have two rows. No disconnected padding is added.

3.2 Exact edit

In native labels the sole change relative to frozen PA194CS is

$$\delta(2066, 1) = (0, L, 302) \quad \rightsquigarrow \quad \delta_F(2066, 1) = (1, R, 21).$$

It changes the written symbol, the head direction and the target selector. The table is `machines/PA194F/machine.tm`; its dense SHA-256 is

`4fcee0c2ef7b917daabf296a55646c730d634f6b5017ea550880b364f2f4afb8.`

PA194F is retained as a particularly transparent intermediate. The stronger fallback PA194R uses a different grammar for exactly this resolved source program, as explained below.

3.3 Clean seed and tape representation

Logical registers $(N, T, W, A, D, H, X, Y, L)$ occupy physical positions $(6, 2, 0, 4, 3, 7, 8, 5, 1)$. The dispatch frame contains a source-address prefix, a variable gap, nine unary register blocks, and a finite nonempty spare list. Value v has a delimiter followed by $v + 1$ ones. The tape outside the finite representation is blank.

PA194F reaches source 25 from blank after 57,768 transitions, with

$$(N, T, W, A, D, H, X, Y, L) = (2, 0, 0, 0, 0, 0, 0, 0, 0).$$

Native dispatch is 84, the head is at -2, header base is 48 relative to the head, and the suffix has 26 spares of value one. There are 66 one cells in the complete tape. The two finite simulators agree on the entire seed, rather than just decoded register values.

Instruction contracts quantify over arbitrary natural registers and arbitrary finite nonempty spare lists. Increment appends a value-one spare; positive decrement shifts the header boundary by one. These effects must be retained to compose segments of the actual tape run.

4 The inherited PA calibration and its audit

The new source instruction does not change the numerical calculus or its command protocol. The ordinary calibration used here is the supplied calibration of the separate-diagonal-head calculus and cached controller. The exact PA conclusion is obtained by the following syntactic chain:

$$\mathrm{Ref}_{\mathrm{PA}} \quad \longleftrightarrow \quad \mathrm{Ref}_{\mathcal{C}} \quad \longleftrightarrow \quad H_{\text{cached source}} \quad \longleftrightarrow \quad H_P. \tag{F3}$$

Here every equivalence is provable in PA. The accompanying reference proofs and their provenance are retained; the following details specify what their use entails and why a mere appeal to external equiconsistency would not suffice. The full ordinary mathematical argument is retained in `references/complete_cached_calibration.tex`; its logical dependencies are retained with the original diagonal and PA195 calibration, and the final source equivalence is detailed in `references/proof_symmetric.tex`. This section audits and summarizes those supplied proofs; it does not claim an independently implemented PA proof-code compiler or a new kernel-checked calibration.

The numerical formula constructors are

$$\begin{aligned} p(a,b) &= \tfrac{1}{2}(a+b)(a+b+1)+b, \\ I(A,B) &= 4p(A,B), \qquad M(a,b) = 4p(a,b)+1, \\ U(a,B) &= 4p(B,a)+2, \qquad E(a,b) = 4p(a,b)+3. \end{aligned}$$

Code zero is truth, code one is the designated contradiction $M(0,0)$, and $N(A) = I(A,1)$. The transposed arguments of U and the alias $I(0,0) = 0$ are part of the specification. The separate head is $\Gamma(B) = I(B, U(B,B))$; the guarded separation gate and the six other heads are those in the retained calibration. The only inference rules are modus ponens and generalization of theorems; the retained dominated implicational schema supplies additional axioms. There is no implicit arbitrary-tautology or truth oracle.

For completeness, write $A \to B$ for $I(A,B)$, and expand conjunction and existential quantification using N and U. The six universally parameterized heads are

$$\begin{aligned} &(\exists x_a\, U(a,C)) \to C, \\ &U(a, U(b,C)) \to U(b, U(a,C)), \\ &M(c,a) \to (E(c,b) \to M(b,a)), \\ &E(c,a) \to (M(b,c) \to M(b,a)), \\ &\exists x_a E(c,a), \\ &U(a,b) \to \big(U(a, b \to c) \to U(a,c)\big). \end{aligned}$$

The dominated schema is

$$((c \to u) \to a) \to ((a \to c) \to (b \to c)), \qquad u \le a.$$

The old guarded gate $G(a,b,c) = N(V \to Q)$ is available when $c \neq a, b$, with the following literal numerical terms:

$$\begin{aligned} V &= M(a,b) \to U(c, M(a,b)),\\ P &= c \to M(c,b),\\ S &= U\big(b, (M(a,b) \to P) \wedge (M(c,b) \to c)\big),\\ Q &= U(c, S \to N(P)). \end{aligned}$$

A failed guard emits truth. The separate diagonal head $\Gamma(c)$ is emitted without the guard. The bundled predecessor emits $N(\Gamma(c) \to (V \to Q))$ instead. Classical implicational proof templates translate the bundled gate to both separate components and conversely; an arbitrary $\Gamma(B)$ is obtained from the bundled gate at $(a,b,c) = (B+1, B+1, B)$, whose guard always holds. These translations are uniform on proof codes. Explosion and double-negation elimination in these templates are derived rules of the actual numerical calculus: explosion follows from the first head and the implicational K theorem, and double-negation elimination then follows from Peirce's law. Treating code one as falsity is therefore supported by derivations, not silently added as a new rule.

PA refutations to numerical refutations. The supplied PA196 proof-producing interpretation uses bidirectional membership rewriting, a neighborhood-equality implication, and an equality witness. The diagonal calculus recovers those same interfaces. For the reverse rewrite

$$C = E(a,b) \to (\chi(b) \to \chi(a)), \qquad \varphi = N(C), \qquad c = \ulcorner \varphi \urcorner,$$

the code c is larger than every variable label occurring in C. The old separation gate at (a,a,c), direct membership rewriting and the retained witness turn the hypothesis φ into the gate's forbidden quantified consequence. The available diagonal clause $\Gamma(\varphi)$ supplies precisely the vacuity needed to generalize that consequence at c. Double-negation elimination yields C. Coincident labels are handled by the separate diagonal membership case. The neighborhood interface is first obtained at the fresh label $\ulcorner E(a,b) \urcorner$ and then transported using the witness and membership-only vacuity. These are finite syntactic proof templates; substitution in the existing refutation compiler is primitive recursive and its correctness is proved by induction on the input proof. This is the first direction in the left equivalence of (F3).

Numerical refutations to PA refutations. The converse is a syntactic interpretation into PA, not a reflection argument. In a fixed conservative definitional extension for exponentiation, interpret membership by the bit predicate

$$B(s,z) \equiv \exists u, v \, \big(s = (2u+1)2^z + v \ \wedge \ v < 2^z\big),$$

equality by equality, and $U(a,A)$ by universal quantification at the corresponding source label. PA proves $B(s,z) \to z < s$, and therefore refutes the translation of code one. For the zero alias use a fixed PA proof of $\top \leftrightarrow (\top \to \top)$; constructor translation is provable equivalence, not unconditional syntactic identity.

Given the syntax of any formula $\theta(z, \vec{u})$, a fixed induction proof pattern produces a PA proof of

$$\forall n \, \exists s < 2^n \, \forall z < n \, \big(B(s,z) \leftrightarrow \theta(z, \vec{u})\big).$$

At the successor step keep the previous code or add 2^n, according to excluded middle for $\theta(n, \vec{u})$. Substituting a formula into this induction instance, choosing fresh auxiliary bound variables, and

checking the resulting finite proof are primitive-recursive syntactic operations. PA verifies the proof-producing operation uniformly; no formula is semantically evaluated by it.

For distinct source labels a, b, the separation witness uses the bounded matrix

$$(B(x_a, z) \vee z = x_b) \wedge T(c)[z/x_b].$$

When the labels $a = b$ coincide, the first membership antecedent under the binder becomes $B(z, z)$ and is false. The correct witness instead uses $(z = x_b) \wedge T(c)[z/x_b]$. This label-collision case cannot be treated as two independent free values. Code freshness makes the output label absent from $T(c)$, and auxiliary arithmetic bound variables are renamed to avoid capture. The remaining heads have fixed first-order proof suppliers. Recursion on a finite numerical proof, followed by the fixed PA refutation of $B(x_0, x_0)$ and elimination of the definitional extension, produces a PA refutation. PA verifies this transformation by induction on the coded proof. This proves the other direction of the left equivalence in (F3).

Refutations and command search. The cached controller maintains a top theorem t, a Cantor-encoded stack w, an untyped offset d, and a cached theorem h. Shift halts exactly when $h = 1$; otherwise it transfers h to the top and pops the next cache. Successful modus ponens puts its consequent in h, clears t, d, and leaves w unchanged. The separate-head AX command pushes ten entries, so its stack selectors use the corresponding ten-entry exponents. The inherited proof compiler produces, from a numerical refutation, a finite command word that halts from *every* incoming store. This last uniformity is essential because earlier search trials do not reset the store. The exhaustive scheduler eventually executes that word unless it has already halted.

In the reverse direction annotate t, h and each live canonical stack entry by a finite numerical derivation. AX appends axiom/generalization proofs, successful MP appends an MP proof, and Shift moves annotated entries or exposes a proof of one. For $w > 0$ the Cantor tail is strictly smaller than w, so the live stack is finite and its annotation is primitive recursively bounded. The alias $p(0, 0) = 0$ is handled by a fixed proof of truth on underflow; it does not require an infinite proof list. The invariant is verified by induction on finite command executions. Together with exhaustive scheduling, this is the middle equivalence of (F3), uniformly on proof and computation codes.

The symmetric Shift. The old cached implementation restores a scratch copy of h and compares $(H, X) = (h, 1)$. The supplied symmetric implementation compares $(1, h)$. The destructive comparator halts exactly at equality and, on inequality, clears both operands while preserving every other register. Consequently both implementations have the same complete surviving store and the same halt condition. The local argument also covers arbitrary incoming scratch: the compared pair is $(h + x, 1)$ or $(1, h + x)$. Its loop induction and finite source binding prove the last equivalence of (F3) in PA.

5 The new source transformation and its PA proof

The new machine PA194F retains the symmetric cached controller of the supplied PA194CS machine and changes the last source instruction of the axiom-emission command. Its only native transition change is

$$(2066, 1) : 0L302 \quad \longmapsto \quad 1R21.$$

The working-state count remains 194. This is a change to the represented instruction and its physical decoder, not a renaming of controls: an unconditional source jump becomes a decrement with two different source successors. The extra branch is unreachable from every intended source start. The proof below establishes that assertion by an invariant involving only four program locations.

The general zero-closing idea is inherited. The supplied 196-state research already replaced the last AX break by a decrement of a register known to be zero, with separate variants for registers 3, 4, 5 and 6; see `TRIT_CONTEXTS.md` in the earlier comparison-and-new196 packet [4]. The new result here is its complete verified composition with the 194-state symmetric cached controller and its particular decoder, together with the four-site proof below. Novelty is claimed for the resulting literal control graph within the pinned corpus, not for the general principle of replacing a jump by a known-zero test.

5.1 The exact four-site invariant

Let P be the 3,646-site source program of PA194CS and let P_F be the source of PA194F. The programs have the same instruction at every site except 3553. A decrement $\operatorname{dec} r[z, p]$ leaves a zero register unchanged and goes to z; on a positive register it subtracts one and goes to p. Put $X = r_6$. The four sites in

$$Z = \{3550, 3551, 3552, 3553\}$$

are as follows, with a decrement's zero successor written first:

site	P	P_F
3550	$\operatorname{jump} 3553$	$\operatorname{jump} 3553$
3551	$\operatorname{dec} r_1[3553, 3552]$	$\operatorname{dec} r_1[3553, 3552]$
3552	$\operatorname{jump} 3551$	$\operatorname{jump} 3551$
3553	$\operatorname{jump} 3637$	$\operatorname{dec} r_6[3637, 3554]$.

Inspection of the complete resolved instruction graph gives exactly two edges entering Z from outside:

3545	$\operatorname{dec} r_6[3551, 3546]$	$3545 \longrightarrow 3551$
3548	$\operatorname{dec} r_6[3550, 3549]$	$3548 \longrightarrow 3550$.

Both are zero branches of a decrement of r_6. This list is an exhaustive finite graph check, not an observation about a sampled run. The two programs have exactly the same outside-to-inside edges. The standalone checker `verification/four_site_cut.py` verifies the complete instruction lists, their sole difference, and this complete boundary without importing either source compiler or symbolic simulator.

Lemma 5.1 (Four-site zero invariant)**.** *For both programs the assertion*

$$\mathrm{pc} \in Z \implies X = 0 \tag{F1}$$

is preserved by every source step. It holds initially at site 25 for arbitrary natural values of all nine registers.

Proof. Site 25 is outside Z, so the initial assertion has no register precondition. For preservation, suppose the next program counter lies in Z. If the current counter is outside Z, one of the two displayed zero branches was taken, and therefore $X = 0$. If the current counter is inside Z, the induction hypothesis gives $X = 0$. Sites 3550, 3551 and 3552 preserve X; the old instruction at 3553 preserves it, and the new instruction there follows its zero branch and preserves it. Thus (F1) holds after the step. A next counter outside Z satisfies (F1) immediately. □

Theorem 5.2 (Exact preservation of source executions)**.** *For every nine-tuple of natural initial register values, the executions of P and P_F from source site 25 have identical configurations at every source time, including identical halting outcomes and halting times. Here a halting configuration may be made absorbing, or equivalently the comparison may stop at the first halt. PA proves this statement for the arithmetized programs.*

Proof. Induct on source time. Away from site 3553 the two instructions are identical. At site 3553, (F1) gives $r_6 = 0$, and both instructions leave the entire register vector unchanged and go to site 3637. This proves agreement of the next complete configurations. Neither program changes the halt instruction.

The instruction lists, their unique difference, the two boundary edges, and the four cases in Z are finite data. The proof is induction on a primitive-recursively coded source run with the formula (F1) as its invariant. PA has that induction instance and verifies the finite instruction lookups. In particular, the argument is an ordinary proof of an internal PA theorem; it does not assume that PA is consistent. □

The source result is stronger than preservation only from the intended seed. In the machine application the seed is

$$(25; 2, 0, 0, 0, 0, 0, 0, 0, 0).$$

The invariant is also considerably smaller than a full nine-register positivity analysis: two entering edges and four source sites suffice.

5.2 Binding the source proof to the binary table

The source theorem would not by itself justify editing a binary table. Three additional bindings are used.

First, the two source resolvers agree on the entire source graph. They identify the old last emitter token as the jump at site 3553 and the new last emitter token as the displayed decrement. All other 3,645 source instructions and all their successor indices agree. The emitter's formula-building instructions, its guarded gate, and its cleanup are unchanged.

Second, the literal table is verified on every primitive source outcome. The new decrement's zero *and positive* outcomes are included, even though (F1) excludes its positive outcome from the intended source run. The contracts quantify over arbitrary natural register values, the variable source gap and an arbitrary finite nonempty spare list. They produce the complete next source frame and a positive number of physical steps. Every intermediate control of a nonhalting instruction contract is nonhalting. The register-list loops used in these contracts are verified from the literal rows; the affine symbolic presentations encode arbitrary run lengths, rather than samples of small register values.

Third, exact finite simulation of the delivered table from the all-zero tape reaches the specified clean frame after 57,768 physical steps, including the head, all nonzero tape cells, the source address and the complete physical register/spare file. The source site is 25 and the logical register vector is the displayed seed. The clean-frame statement includes the zero exterior of the finite tape support.

Lemma 5.3 (Positive finite simulation and converse halting)**.** *PA proves that the delivered binary table halts from blank if and only if P_F halts from its clean seed.*

Proof. Concatenate the positive physical instruction segments after the fixed startup. A finite source halt supplies a finite literal halt.

For the converse, suppose a literal halting trace has length T. If $T < 57{,}768$, it contradicts the checked nonhalting startup by determinism. Otherwise the startup precedes that halt. Suppose the source has no halt during its first $T + 1$ instructions after the seed. The source machine's increment, decrement and jump operations are total. Their physical contracts can therefore be concatenated for all $T + 1$ source instructions. Their lengths are positive, so the resulting physical run extends beyond time T. Its intermediate states are nonhalting. This contradicts determinism of the given literal halting trace. Thus the source halts within a bounded finite source trace.

This proof is arithmetical. A source register grows by at most one at each primitive source instruction; finite source iteration and its encoding are primitive recursive. The physical contract proofs use terminating unary sweeps, fixed finite row calculations and induction on the relevant natural lengths. Concatenation and trace determinism are proved by induction on coded finite sequences. These are PA proofs, including for nonstandard trace lengths in a model of PA. No reflection principle or standard-model soundness argument is used. □

This gives

$$\mathrm{PA} \vdash H_{\mathrm{PA194F}} \longleftrightarrow H_{P_F} \longleftrightarrow H_P. \tag{F2}$$

The two binary tables can consume different physical times and spare lists. Equality of their source configurations is the relevant invariant.

Theorem 5.4 (PA194F consistency calibration)**.** *Let H_{PA194F} be the arithmetic sentence asserting a finite halting trace of the delivered table from its specified blank start. For a fixed ordinary primitive-recursive PA proof predicate,*

$$\mathrm{PA} \vdash H_{\mathrm{PA194F}} \leftrightarrow \exists p\ \mathrm{Prf}_{\mathrm{PA}}(p, \ulcorner 0 = 1 \urcorner),$$

and therefore

$$\mathrm{PA} \vdash \neg H_{\mathrm{PA194F}} \leftrightarrow \mathrm{Con}(\mathrm{PA}).$$

Proof. Compose (F2) with the proof-producing equivalences in (F3), and negate the PA-refutation predicate. All transformations and the inductions establishing their correctness are available in PA. The biconditional itself does not assume PA consistency; actual standard-model nonhalting follows under that assumption. □

Precise proof status. This is an ordinary mathematical proof of a PA-provability statement, with the unchanged numerical calibration explicitly attributed to and retained from the supplied research. The new four-site invariant and source equivalence are proved above; the literal-table bindings are supported by complete finite and universal symbolic checks. The latter use the mathematical soundness of the inherited symbolic rules and their register-list induction lemmas, as well as the recorded Python executions; a successful execution is not itself a kernel proof. No new end-to-end Lean or Rocq theorem is claimed. In particular, neither the upstream PA196 one-direction formal theorem nor the conditional iPRA endpoints are relabeled as a kernel-checked proof of this biconditional.

6 PA194R: a reassociated 194-state decoder

PA194R combines the final-emitter decrement proved above with a different binary association of the axiom-emitter program. It has 194 working states and 388 transition rows. The complete primitive instruction graph is identical to PA194F, but 1,111 of its 3,646 source addresses change. These addresses are the binary paths physically processed by the decoder, so the construction changes the encoded program and its shared decoder graph. It is more than a relabeling of PA194F's controls. The known-zero closing principle and symmetric cached controller remain the explicitly attributed inherited components.

Of the 67 axiom-reader subtree types, 23 are shared with PA194F and 44 are replaced by different types. The maximum complete source-address length remains 37. Thus the claim of a different decoder is supported both by its changed sharing structure and by the separate control-graph isomorphism check, not merely by a different printed numbering.

The new association retains 67 distinct axiom-reader states before folding. Together with the 135 fixed controls, the unfolded machine has 202 working states. Seven contextual folds in the axiom reader and the retained symmetric-controller fold give

$$135 + 67 - 7 - 1 = 194.$$

The physical state savings therefore survive the changed program association. The construction and fold records are reproducible from `construction/build_reassociated.py`; the final machine and its source paths are retained in `machines/PA194R/`.

The dense table's SHA-256 is

`8a9a8a30773f73b638a7c51da1afca9c9921371aea819d6a21f02d742f673a4e,`

and the native table's SHA-256 is

`24539448def6e974d636052dbcb56eeb61d8e06ad0e9924d734c3bd4f0e01aa8.`

6.1 Exact source equality and independent literal verification

Preserving a flattened instruction word would not be sufficient here: breaks and continues depend on the surrounding binary program structure. The independent verifier instead parses the delivered source paths and resolves every successor twice, using the arithmetic-path and interval algorithms. The results agree at all 3,646 sites, and agree exactly with PA194F's resolved graph, including both targets of every decrement. The primitive operation and register operand also agree at every site. Thus the change is in physical source addresses and decoding; it makes no change to the already-proved primitive source machine P_F.

The standalone verifier `verification/check_reassociated.py` reads the delivered native transition table directly. It does not import the construction compiler or use an arithmetic-register oracle. It rederives the four register-list lemmas from the literal rows and checks all 4,746 primitive outcomes, including the positive branch at the final emitter decrement. Each contract quantifies over arbitrary natural register values, an arbitrary source gap, and an arbitrary finite nonempty list of arbitrary natural spare values. The contracts certify complete frame transformations and positive physical durations, with no intermediate halt in a nonhalting instruction segment.

A separate simulator stores the finite set of tape cells containing one, so the exterior is exactly the infinite zero tape. From native start control 74 it reaches the complete clean source frame after 57,768 transitions: native control 84, head -2, source site 25, logical registers $(2, 0, 0, 0, 0, 0, 0, 0, 0, 0)$, and the recorded 26 spare registers. The entire finite tape support is compared with the frame encoding, not just the decoded logical registers. The source path at site 25 is unchanged by reassociation. The dense table is checked to be the exact breadth-first renaming of the native table, with dense start 0 and halting target 194.

Receipts, all symbolic contracts, the rederived list lemmas, the complete blank seed and the list of changed source addresses are retained in `verification/recorded/PA194R/`. The verifier supports `-output` for replay into a fresh directory and rejects Python's optimized mode, which would disable its assertions.

Theorem 6.1 (Consistency calibration of PA194R)**.** *For the delivered binary table with its specified blank start,*

$$\mathrm{PA} \vdash H_{\mathrm{PA194R}} \iff H_{P_F} \iff \exists p\ \mathrm{Prf}_{\mathrm{PA}}(p, \ulcorner 0 = 1 \urcorner).$$

Consequently

$$\mathrm{PA} \vdash \neg H_{\mathrm{PA194R}} \iff \mathrm{Con}(\mathrm{PA}).$$

Proof. Exact graph equality identifies PA194R's source machine with P_F. Its independently checked startup supplies the same logical seed. Concatenating the total positive instruction contracts maps a finite source halt to a finite literal halt. Conversely, a literal halt at time T cannot coexist with $T+1$ nonhalting source instructions: their positive physical segments, preceded by startup, would extend a deterministic nonhalting literal execution beyond time T. This is the same bounded converse argument established for PA194F. The source equivalence and inherited syntactic PA calibration then give the displayed biconditionals. All finite checks and run inductions have the ordinary mathematical PA-provability scope explained in the preceding sections. □

The changed association need not preserve physical running times or spare-list lengths. Each complete-frame contract accounts for its own effects; source equality is what transfers the logical calibration. As with PA194F, the new result is supported by ordinary proofs and complete symbolic verification, with the inherited calibration and symbolic-rule soundness dependencies stated explicitly. No new end-to-end proof-assistant kernel theorem is asserted.

7 Two companion 194-state compositions

PA194A and PA194X instead replace the jump at source 26, leaving the emitter's final return unchanged:

$$\begin{aligned} \text{jump}\,34 &\rightsquigarrow \text{dec}\,r_3[34,27] \quad \text{(PA194A)},\\ \text{jump}\,34 &\rightsquigarrow \text{dec}\,r_6[34,27] \quad \text{(PA194X)}. \end{aligned}$$

Native row $(27,1)$ changes from $0L40$ to $1R17$ or $1R21$. The idea is related to supplied PA194CZ, but these compositions use the 3,646-site symmetric controller, rather than CZ's 3,649-site controller. Their graphs are distinct from the prior eight classes.

For PA194A the zero cut is

$$Z_A = [0,9] \cup [11,26] \cup [3523,3645], \qquad |Z_A| = 149.$$

Intervals are inclusive. Its only outside entries are zero branches of decrement A at $48 \to 3638$, $49 \to 3554$ and $3519 \to 3523$. No cut instruction increments A. The intended seed lies in the cut with $A = 0$; thus $pc \in Z_A \Rightarrow A = 0$ is inductive.

For PA194X use

$$Z_X = \{25, 26, 3550, 3551, 3552, 3553, 3633, 3634, 3635, 3636, 3637\}.$$

Its only outside entries are zero branches of decrement X at $22 \to 25$, $3545 \to 3551$, $3548 \to 3550$, $3628 \to 3634$ and $3631 \to 3633$. No cut instruction increments X, and the seed has $X = 0$. The changed jump therefore has exactly its old effect.

The checker examines every incoming edge of both complete graphs. Each companion has 194 controls, 3,646 sites and 4,746 outcomes. They reach the same source seed after 57,978 and 58,014 transitions. Physical times may differ; source preservation and positive simulation give the same ordinary PA calibration.

8 Verification: what was established

8.1 Two implementations and their boundaries

The construction checker rebuilds source and table, checks all folds together, and discharges every primitive runtime contract. It uses inherited arithmetic-block lemmas on blocks whose rows are explicitly

checked unchanged. This is useful, but it is not the only evidence.

The independent literal checker does not import the compiler or use arithmetic-box oracles. It resolves source graphs in two ways, rederives four generic register-list loop lemmas from literal rows, and verifies all outcomes with an unbounded symbolic tape. Affine run lengths denote arbitrary natural values. Generic loops are justified by induction; the finite checks instantiate and compose those symbolic facts, rather than testing a finite range of numerical register values.

Check	PA194R	PA194F	PA194A	PA194X
Working states / rows	194 / 388	194 / 388	194 / 388	194 / 388
Primitive source sites	3,646	3,646	3,646	3,646
Universal outcomes	4,746	4,746	4,746	4,746
Construction symbolic steps	1,518,961	1,514,002	1,513,882	1,513,882
Independent symbolic steps	1,104,215	1,100,612	1,100,478	1,100,566
Maximum independent steps/case	409	403	403	403
Minimum duration constant	3	3	3	3
Blank startup transitions	57,768	57,768	57,978	58,014
Small zero-cut sites	4	4	149	11

Table 3: Completed checks. Symbolic steps count checker operations, not actual run time for all numerical inputs.

Independent output comparison confirms identical native and dense bytes and identical complete source graphs, site lists and label maps between the two implementations. All outcome contracts and loop certificates are included. A larger 263,180-pair positivity closure was also checked for the direct variants; the small cuts suffice for the written source proof.

The trusted boundary remains explicit: ordinary mathematical proofs supported by executable symbolic certificates, with Python and the inspected symbolic rules in the verification base. They are not proof terms checked by a small formal kernel. The archive retains the inherited logical proof texts, frozen source inputs and licenses. No new claim about either machine-specific iPRA direction is made.

8.2 A useful rejected construction

Changing native $(103, 1)$ from $0L302$ to $1R21$ replaces eight closing source jumps by known-zero decrements of X. All 4,753 source outcomes pass the construction contract checker, and the source invariants hold. Nevertheless the literal table halts from blank after 1,361 transitions, at head position 1, before reaching the intended seed. Maximum live support is only 26 ones; the seed requires 66.

This is a rejection, not another consistency machine. The same physical control participates in startup and source decoding. Canonical-source contracts do not justify changing its startup behavior. The full trace, table and independent replay are under `research/rejected103/`. This negative control explains the separate startup proof obligation.

9 Search below 194 states

The unsuccessful primary target remains reviewable through supplied source, bounds, inputs and logs. No lower-bound theorem is claimed.

The new local quotient synthesizers jointly solve for both rows of a retained state while identifying another state with it. They permit internal revisits to the retained state, unlike a no-reentry filter. A

second version allows old and new paths to converge after positive finite segments. A known fold is recovered as a positive control.

Radius-six, 200-step joint-return searches ran on all seven nonrejected prior 194-state classes in native form and on new compositions. Each machine run examines 18,721 unordered control pairs; none returns a surviving quotient. Supplied symbolic-context variants and radius-four, 100-step two-sided convergence probes likewise produce no accepted reduction. Logs distinguish action trials, candidate row pairs and matches.

Grammar searches score readers minus compatible folds, not merely source tokens. All $9! = 362{,}880$ physical register bijections were screened for the fixed AX shape without improvement. Four final-zero endpoints (A, D, H, X), two frontier shapes and 46,656 parameter-role assignments per combination give 373,248 cases; all remain at least 194. The final three bounded randomized reassociation runs made 16,903,868 proposals and found no sub-194 candidate. An earlier saved plateau supplied the new PA194R association; it was then subjected to complete construction and literal checks.

These searches exclude nonlocal tape recodings, changes to several interpreter controls beyond the tested templates, longer local simulations and different proof enumerators. A failed bounded search does not show that another state cannot be removed.

10 Ideas from ZFC and ConciseIPRA

The ZFC work is useful at the compilation and proof-engineering layers [6]: its reductions combine source cleanup, tree sharing, contextual folds and exact startup checks. That motivates scoring source DAGs together with physical interpreters, allowing contextual quotient paths, and preserving whole register/spare frames. Sharing an open subtree can require a compensating decoder or scanner control, so a source saving is not automatically a state saving.

A direct attempt to change the PA spare-creation row to a zero-valued spare convention failed literal blank startup. This tested edit gives no 193-state machine; it would need a new codec, startup and refinement argument. Nor can a ZFC consistency machine be relabeled as a PA consistency machine: the refutation predicates differ.

ConciseIPRA offers a different source-language direction: very small complete proof checkers with explicit substitution, freshness and witness rules [7]. Its metrics are normalized program tokens, AST nodes and bytes, not binary TM states. A short checker still needs a parser, enumerator, data encoding and compiler before its physical cost is known.

Its advertised theory is bi-interpretable with iPRA. Searching for a contradiction in that calculus is calibrated to that calculus, not automatically to PA. Using it here requires the appropriate PA-strength proof/refutation translations and internalized computation. None of the delivered tables silently changes the target to Con(iPRA).

The concrete next direction is to optimize the axiom word after exposing several proved known-zero returns while designing compatible startup at the same time. The final-X word is now a verified seed; the row-103 failure shows why replacing every similar return is insufficient.

11 Using and reproducing the archive

Python 3.10 or later and its standard library suffice for builders and verifiers. Keep assertions enabled: do not use `-O` or `PYTHONOPTIMIZE`. No network is needed. Search programs additionally need a C++17 compiler. From the extracted archive root:

```
python3 verify.py
python3 verify.py --full
```

The first checks the manifest when present, rebuilds the machines in a temporary directory with construction contracts, checks zero cuts, compares delivered and independent outputs, and replays the rejected startup. The second reruns every independent literal symbolic check, including derivation of the four list lemmas. It takes several minutes.

Individual entry points are `construction/build.py`, `construction/build_reassociated.py`, `proofs/check_zero_cuts.py`, `verification/four_site_cut.py`, `verification/check_literal.py`, `verification/check_companions.py` and `verification/check_reassociated.py`. Tables use rows `statereadwritedirectiontarget`. Appendix A prints the primary dense table in full.

Comparison tools use the saved census or an explicitly supplied checkout. Bundled old tables permit novelty checks without fetching archives. CSV/JSON occurrence paths describe the pinned snapshot; they are not dependencies needed to verify the new machines. To compile this report run twice:

```
cd report
pdflatex -interaction=nonstopmode -halt-on-error main.tex
```

Standard packages listed in the preamble are required. The PDF has been rendered and visually checked. Licenses and attribution are retained under `licenses/` and `references/`.

A Complete dense transition table for PA194R

Start is 0; halt is 194. The initial tape is all zero and the head starts at zero. An entry wDq means write w, move D and enter q. State 194 has no rows and is not counted.

q	read 0	read 1	q	read 0	read 1	q	read 0	read 1	q	read 0	read 1
0	1R1	1R2	49	0L66	0L15	98	0R112	1R104	147	0R149	1R142
1	1R3	1R1	50	0R67	0L10	99	0R113	0L14	148	0R158	1R46
2	0R0	1R2	51	0R68	1R69	100	0R114	1L66	149	0R91	1R159
3	0R4	0R1	52	0R70	1R71	101	0R115	1R116	150	0R160	1R161
4	1R5	1L6	53	0R72	1R73	102	0R117	1R102	151	0R162	1R41
5	1L7	1L8	54	0R74	1R75	103	0R14	1R103	152	0R163	1R164
6	0L7	1L6	55	0L76	0R1	104	0L109	1L33	153	0R165	1R157
7	0L9	1L6	56	0R61	1R56	105	0R67	1R118	154	0R166	1R156
8	0L10	0L11	57	0L68	1L35	106	0R119	1L120	155	0R135	1R130
9	1R12	0L9	58	0R77	1R78	107	0R121	1R122	156	0R147	1R40
10	0L9	0L10	59	0R51	1R79	108	0R46	0L14	157	0L155	1L167
11	1L8	1L11	60	0L80	1R81	109	0R85	1R94	158	0L168	1R169
12	0R12	0R13	61	0R82	1R61	110	0R109	1L89	159	0R100	1R79
13	0L14	0R13	62	0L76	0R2	111	0R123	1R123	160	0R170	1R168
14	1L15	0L15	63	0R83	1R63	112	0L60	0R82	161	0R171	1R172
15	0L15	1L16	64	0R30	1R84	113	0R62	0R117	162	0R173	1R174
16	0L17	1L16	65	0R85	1R86	114	0R55	0R117	163	0R175	1R158
17	0L18	1L16	66	0R35	1R87	115	0L80	1R124	164	0L176	1L177
18	0L19	1L16	67	0R41	0L13	116	0R125	1R126	165	0R149	1R140
19	0L20	1L16	68	0R88	1R65	117	0R127	1R117	166	0R164	1R178
20	0L21	1L16	69	0R47	1R89	118	0R47	1R84	167	0R179	1R180
21	0L22	1L16	70	0R41	1R90	119	0R55	0R127	168	0R181	1R145
22	0L23	1L16	71	0R91	1R92	120	0R35	1R33	169	0R179	1R130
23	0R23	1R24	72	0R93	1L94	121	0R93	1R108	170	0R139	1R164
24	0R25	1R26	73	0R47	1R72	122	0R74	1R73	171	0R179	1R182
25	0R27	1R28	74	0R87	1R95	123	0R46	1R46	172	0R183	1R100
26	0R29	1R30	75	0R47	1R96	124	0R128	1R129	173	0R31	1R49
27	0R31	1R194	76	1R76	1L15	125	0R32	1R64	174	0R184	1R173
28	0L32	1R33	77	0R88	0L7	126	0R31	1R106	175	0R134	1R185
29	0R34	1R35	78	0R97	1R98	127	0R63	1R127	176	0R144	1R186
30	0L33	1L36	79	0R41	1R99	128	0R130	1R131	177	0R167	1R187
31	0R37	1R38	80	0R100	1R84	129	0R132	1R133	178	0R40	1R188
32	0R39	1R40	81	0R101	0L67	130	0R134	1R135	179	0R40	1R40
33	0R41	1R42	82	0R102	1R82	131	0R136	1R137	180	0R149	1R189
34	0R43	1R44	83	0R103	1R83	132	0R138	1R139	181	0R180	1R46
35	0R45	1R36	84	0R104	0L14	133	0R140	1R141	182	0R153	1R146
36	0R46	1R47	85	0R62	0R102	134	0R91	1R120	183	0R164	1R177
37	0R48	1R41	86	0R105	0L84	135	0R71	1R142	184	0R118	1R106
38	0R49	1R50	87	0R30	1R57	136	0R143	1R144	185	0R157	1R190
39	0R51	1R52	88	0L60	0R83	137	0R145	1R146	186	0R191	1R154
40	0R53	1R54	89	0R88	1L106	138	0R134	1R147	187	0R176	1R192
41	0R55	0R56	90	0R95	0L14	139	0R46	1R148	188	0R189	1R179
42	0R57	0L14	91	0R64	1R107	140	0R149	1L131	189	0L175	1R46
43	0R58	1R59	92	0R106	1R70	141	0R150	1R151	190	0R193	1R144
44	0R60	1R49	93	0L60	0R103	142	0R40	1R123	191	0R192	1R135
45	0R55	0R61	94	0L59	1R108	143	0R28	1R152	192	0R189	1R40
46	0R62	0R63	95	0L109	1R110	144	0L153	1L154	193	0R32	1R71
47	0L64	1L65	96	0R111	0L14	145	0L129	1R155			
48	0R45	1R50	97	0R112	1R89	146	0R156	1R157			

B All 41 research packets in the comparison

This ledger records reported contributions at the snapshot, rather than upgrading every experiment to a theorem. Except for the five newest incoming archives, names refer to subdirectories of `Computability/BusyBeaver/Research/PA/docs/`. Exact paths and hashes are in `comparison/packet_ledger.csv`.

Packet	Reported contribution
1. `pa_unary_power_cover`	Unary power covering at PA strength (report of 27 October 2026). The U208 packets carry it as a baseline.
2. `U208_ConPA_Conditional_Context_2026-10-29`	U208 by a conditional tape-context reduction. States a 208-state candidate machine/u208.tm.
3. `U208_ConPA_Entry_Context_2026-10-29`	U208 by eliminating a state using entry context. States a 208-state machine machine/u208.tm.
4. `U208_ConPA_History_2026-10-29`	U208 by a finite-history decoder fold. States a 208-state candidate machine/u208.tm.
5. `U208_ConPA_2026-10-29`	U208 by a three-cell fold. States a 208-state candidate machine/u208.tm.
6. `u208_research_2026-10-29`	U208 research release. States two 208-state machines, machine/u208.tm and machine/variant_plain/u208.tm.
7. `Below_U208_Research_2026-10-29`	Research toward fewer than 208 states. States that no machine with 207 states or fewer was obtained.
8. `u208_comparison_research_2026-10-29`	Comparison of the 208-state proposals and a search below 208. States that the target was not achieved.
9. `ConPA_below208_research_2026-10-29`	Comparative audit and reduction search below 208. States that it does not establish a 207-state machine.
10. `pa_consistency_research_2026-10-29`	Audit and new source reductions. States that no validated machine below 208 states was obtained.
11. `pa208_research_2026-10-29`	Comparison and research below 208 states. States that it does not establish a valid machine below 208.
12. `pa_tm_research_2026-11-13`	Comparison and reduction research. States a new 196-state variant with shorter source paths and nothing below 196.
13. `pa_consistency_research_2026-11-13`	Comparison and diagonal-vacuity research. States a shorter axiom program with a 199-state implementation and nothing below 196.
14. `conpa_tm_comparison_and_reduction_search`	Comparison and reduction search. States two experimental 196-state command variants and nothing below 196.
15. `pa196_research_2026-11-13`	PA196 comparison, source factorization and proof obligations. States a source tree of 157 internal nodes and nothing below 196.
16. `pa_consistency_research`	Research and verification packet. States that no machine below 196 states has been validated.
17. `pa_tm_comparison_and_reductions`	Comparison and new rewrites, a research snapshot.
18. `pa_tm_extended_research_2026-11-13`	Comparison and refined reduction search. States that the smallest accepted count is 196.
19. `conpa_tm_comparison_and_search`	Comparison and a search below 196. States that none was obtained; files named 195 are rejected or diagnostic candidates.
20. `pa_consistency_reduction`	Comparison and reduction experiments. States two new 196-state variants and a shorter source program with a 203-state implementation.
21. `pa_consistency_comparison_2026_11_13`	Comparison and new reduction experiments. States that nothing below 196 was obtained.
22. `PA_consistency_TM_research_2026-11-13`	Comparison and new source reductions. States that no machine below 196 is claimed.
23. `pa_consistency_research_2026-11-13_2`	Comparison and search below 196. States two new layouts with 196 and 198 states and a command-level reduction.
24. `conpa_tm_study_2026-11-13`	Comparison and reduction study. States that nothing below 196 was obtained.
25. `pa_below196_research`	Comparison and reduction experiments. States that nothing below 196 was obtained.
26. `pa_consistency_comparison_and_new196_2026-11-13`	Comparison and reduction research. States that it contains no accepted machine below 196.

Continued on next page

Packet (continued)	Reported contribution
27. `pa_tm_research_2026-11-13_2`	Comparison and reduction investigation. States same-size work, failed candidates and nothing below 196.
28. `pa_consistency_cross_archive`	Cross-archive constructions and reduction search. States that it does not certify a 195-state machine.
29. `pa195_conpa_2026-11-13`	PA195. States a 195-state machine pa195/pa195.tm and an article with a PA proof of the equivalence.
30. `pa_consistency_reduction_audit_2026-11-14`	Comparison and reduction audit. States nothing below 196 and no new theorem.
31. `pa_consistency_research_2026-11-14`	Comparison and NEG18. States an 18-instruction encoded negation with a compiled machine of 197 states.
32. `PA195_ConPA_research_2026-11-14`	A 195-state machine. States the primary machine_195S.tm and a second table machine_195C.tm.
33. `pa195_followup_research`	Comparison and searches below 195. States a graph-distinct 195-state machine M195S8 and nothing below 195.
34. `pa194_conpa_2026-11-13`	A 194-state machine. States the table construction/tables/pa194.tm and a proof of the equivalence in PA.
35. `pa194_conpa_2026-11-13_2`	PA194. States a 194-state machine pa194/pa194.tm and an article with a PA proof of the equivalence.
36. `pa195_diagonal_combined_2026-11-13`	PA195DC. States a different 195-state machine.
37. `pa194C_conpa_2026-11-13(2)`	PA194C cached-head controller on the 67-reader/seven-fold decoder. Same authoritative table d759313c as four other new packets; no sub194 result.
38. `pa194c_conpa_2026-11-13`	PA194C cached-head controller on the 67-reader/seven-fold decoder. Same authoritative table d759313c; no sub194 result.
39. `pa194c_conpa_2026-11-13(1)`	PA194C plus supplementary PA194CZ invariant-based zero-register jump. CZ is a distinct 194-state construction seed with construction checker and invariant, no independent physical replay.
40. `pa194c_conpa`	PA194C plus audited PA194altC search input using the alternate 68-reader/eight-fold allocation. The latter has complete inherited-engine source/contract/startup checks, not a newly independent lemma derivation.
41. `pa194_new_designs_2026-11-13`	PA194C plus PA194CS, the symmetric-Shift guard construction: 3646 source sites, one new reader balanced by a contextual decrement-interface fold. No sub194 result.

References

[1] V. Reshetnikov, *ProveIt: PA consistency machines*, revision `17f92105e2bc4317feae6c219cb2143437b482ff`. https://github.com/VladimirReshetnikov/ProveIt/blob/17f92105e2bc4317feae6c219cb2143437b482ff/Computability/BusyBeaver/Research/PA/README.md.

[2] *PA194 construction and calibration*, supplied packet `pa194_conpa_2026-11-13` and alternate `_2`, same revision. https://github.com/VladimirReshetnikov/ProveIt/tree/17f92105e2bc4317feae6c219cb2143437b482ff/Computability/BusyBeaver/Research/PA/docs.

[3] *PA194 cached and symmetric designs*, supplied `pa194_new_designs_2026-11-13.zip` and `pa194C_conpa_2026-11-13(2).zip`, under `docs/incoming`. Relevant mathematical fragments are retained in `references/`. https://github.com/VladimirReshetnikov/ProveIt/tree/17f92105e2bc4317feae6c219cb2143437b482ff/docs/incoming.

[4] *Four zero-closing variants*, `TRIT_CONTEXTS.md`, prior comparison-and-new196 packet. https://github.com/VladimirReshetnikov/ProveIt/blob/17f92105e2bc4317feae6c219cb2143437b482ff/Computability/BusyBeaver/Research/PA/docs/pa_consistency_comparison_and_new196_2026-11-13/engine/TRIT_CONTEXTS.md.

[5] Supplied ordinary logical calibration and preservation proofs, retained with attribution: `references/original_pa195_calibration.tex`, `references/diagonal_calculus.tex`, `references/separate_head_calculus.tex`, `references/complete_cached_calibration.tex`, `references/proof_cached.tex` and `references/proof_symmetric.tex`. Their ordinary PA claims differ from the formal status in reference [1].

[6] *ProveIt: ZFC consistency machine research*, same revision. https://github.com/VladimirReshetnikov/ProveIt/blob/17f92105e2bc4317feae6c219cb2143437b482ff/Computability/BusyBeaver/Research/ZFC/README.md.

[7] *Concise presentations of intuitionistic primitive recursive arithmetic*, same revision. https://github.com/VladimirReshetnikov/ProveIt/blob/17f92105e2bc4317feae6c219cb2143437b482ff/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/README.md.